Práctica 2



Ejercicio 0

a) Codifica en Python puro (solo módulos estándar de Python) los algoritmos de **Needleman-Wunsch** (alineamiento global) y **Smith-Waterman** (alineamiento local).

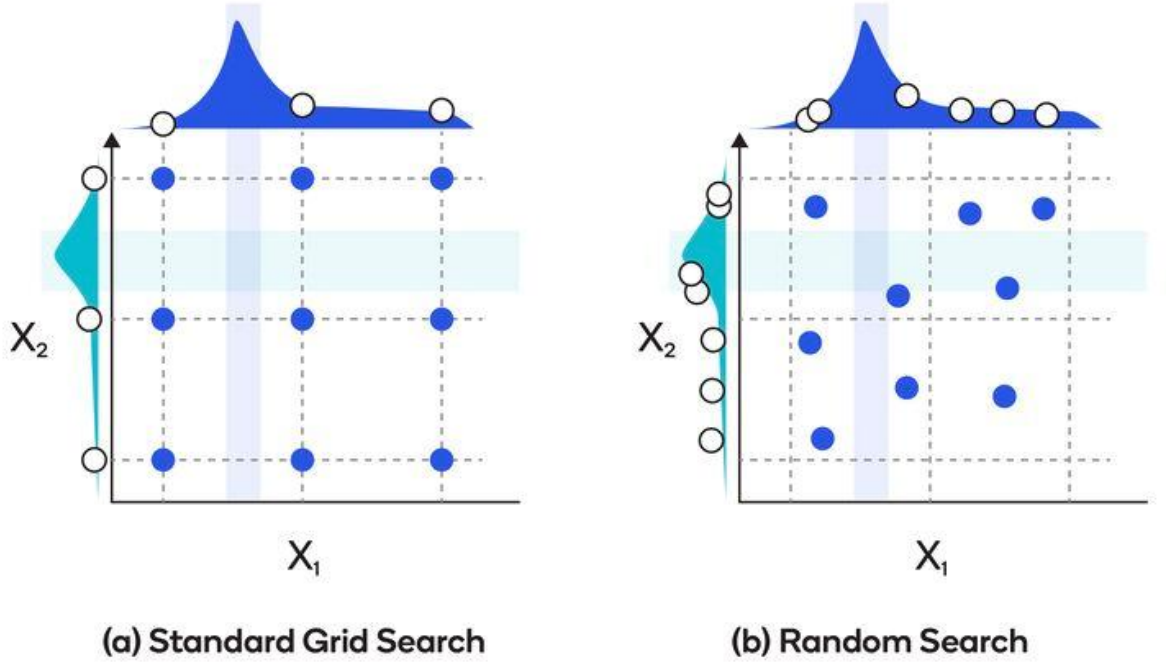
b) Luego lleva a cabo varios alineamientos globales y locales para poner a prueba los dos algoritmos mostrando sus resultados, comparándolos con la solución de Biopython y comentándolos.

Las cadenas pueden ser de nucleótidos o de aminoácidos y puedes generarlas manual o aleatoriamente. Para examinar los resultados se recomienda que no sean demasiado largas (de longitud 10 o menos)]

NOTA: Los algoritmos están explicados en la presentación.

Ejercicio 1

En *Machine Learning*, para buscar los parámetros óptimos de un modelo se puede usar *Grid Search*. Aquí, el alineador es el modelo, y las penalizaciones de huecos son los hiperparámetros.



(a) Standard Grid Search (b) Random Search

```
# Secuencia original de un gen sano
seq_sana      = "GCATGCATGCATGC"

# Secuencia de un gen degradado (paciente)
seq_paciente = "GCATGCGC"
```

Crear dos bucles anidados para iterar sobre un espacio de búsqueda: open_gap (de -1 a -20) y extend_gap (de -0.1 a -5.0). Elegir el intervalo.

Para cada combinación, ejecutar el PairwiseAligner sobre las secuencias original y mutada.

NOTA IMPORTANTE: Convertir un concepto biológico en un problema de optimización puro es **peligroso**.

Almacenar el número de "empates óptimos" (tamaño del iterador) generados por cada combinación en un DataFrame de Pandas.

Descubrir matemáticamente en qué punto exacto del espacio paramétrico el algoritmo "colapsa" y pasa de dar múltiples empates a dar una única solución óptima. Muestre la solución obtenida.

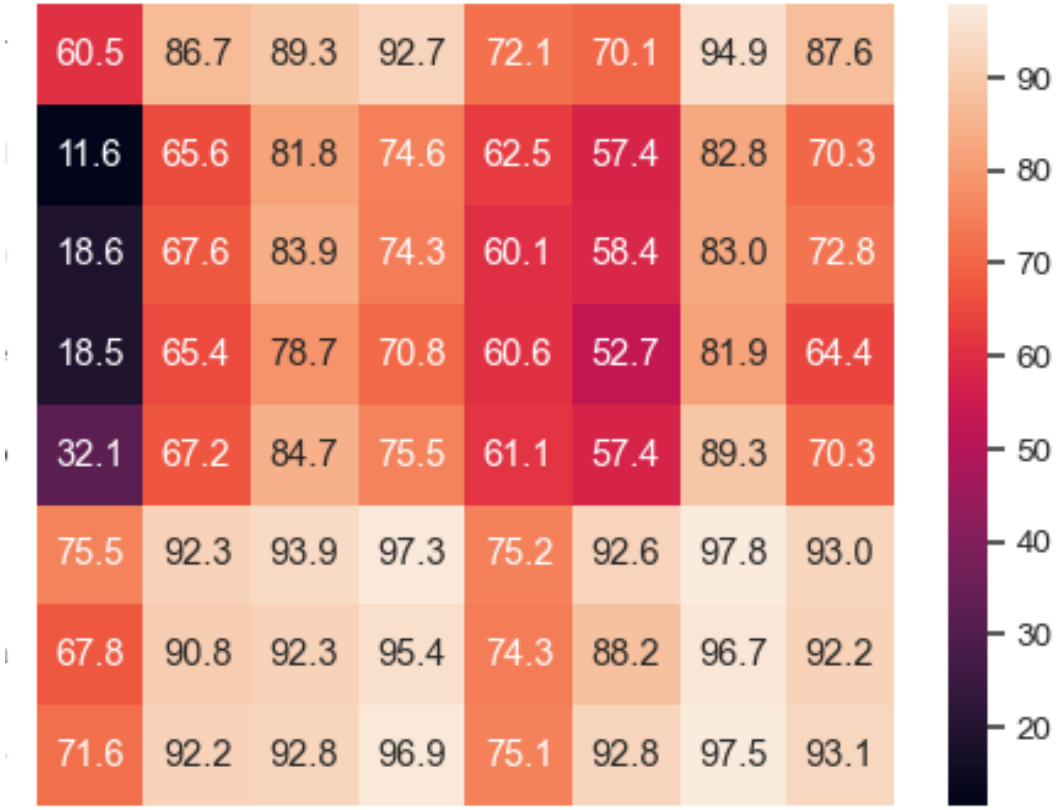
Haga alguna aportación adicional.

NOTA IMPORTANTE: Convertir un concepto biológico en un problema de optimización puro es **peligroso**.

Ejercicio 2

Cargar un archivo FASTA con 5 secuencias de la misma proteína (ej. Humano, Chimpancé, Ratón, Pez, Mosca) usando Bio.SeqIO.

Programar un bucle de "todos contra todos" para calcular el .score de alineamiento global usando BLOSUM62 entre cada par de especies.



Volcar esas 25 puntuaciones en una matriz de NumPy o un DataFrame de Pandas.

Renderizar esa matriz usando un *Heatmap* de la biblioteca Seaborn o Matplotlib.

Experimentar con otras matrices de substitución.

Comentar los resultados.



Ejercicio 3

La programación dinámica es perfecta pero costosa. Un ingeniero de datos necesita saber si su pipeline escalará cuando pase de procesar 100 datos a procesar 1 millón.

Escribir un script que genere secuencias de ADN aleatorias (usando la librería random de Python) de tamaños incrementales: 100, 500, 1000, 2000 y 5000 bases.



Usar un módulo de tiempo de alto rendimiento para cronometrar en milisegundos cuánto tarda PairwiseAligner en alinear cada par.

Graficar (con Matplotlib) la longitud de la secuencia en el eje X y el tiempo de ejecución en el eje Y.

Interpretar el resultado obtenido.

